

Entrega Projetos 3 - RSD

Atividade Avaliativa — Diagrama de Arquitetura da Aplicação

Ferramenta obrigatória: draw.io (diagrams.net)

Entrega: documento em PDF + arquivo-fonte `.drawio` **Valor:** 3,0 pontos

1. Objetivo

Representar graficamente a arquitetura da aplicação que vocês estão desenvolvendo no projeto integrados, de modo que **outra pessoa consiga entender como o sistema funciona sem ler o código**.

O diagrama deve responder a três perguntas:

1. **Quem fala com quem?** (componentes e ligações)
2. **Como cada um fala?** (protocolo e porta de cada ligação)
3. **Onde cada um mora?** (redes, sub-redes e o que é público ou privado)

Além disso, os **endpoints REST** expostos pela aplicação devem estar documentados e coerentes com o desenho.

2. O que deve ser entregue

Um único documento PDF contendo, nesta ordem:

#	Seção	Conteúdo
1	Capa	Nome do sistema, integrantes, turma, data
2	Descrição do sistema	Máximo 1 página: o que o sistema faz e quem são os usuários
3	Diagrama de arquitetura	Exportado do draw.io, legível, com legenda
4	Tabela de endpoints REST	Seção 4.2 deste documento
5	Tabela de ligações e protocolos	Seção 4.3
6	Tabela de definição de redes	Seção 4.4
7	Legenda e convenções	Significado de cores, formas e tipos de linha

E, junto do PDF, o **arquivo `.drawio` original** (não vale só a imagem — preciso conseguir abrir e editar).

Nomeação dos arquivos: `arquitetura-<nome-do-projeto>-<sobrenome1>-<sobrenome2>.pdf` e `.drawio`

3. Passo a passo

Etapa 1 — Inventário de componentes (faça antes de abrir o draw.io)

Em uma folha ou planilha, liste **tudo que executa código ou guarda dado** no seu sistema. Use estas perguntas como guia:

- Quem inicia as requisições? (navegador, app mobile, outro sistema, job agendado)
- Existe frontend separado do backend? Ele é servido de onde?
- Quantos serviços de backend existem? Um monolito ou vários?
- Quais bancos de dados? Relacional, cache, busca, arquivos?
- Existe fila ou mensageria? (RabbitMQ, Kafka, SQS)
- Existe integração com terceiros? (gateway de pagamento, e-mail, mapas, API de CEP)

- Existe algo de infraestrutura no caminho? (load balancer, API gateway, CDN, proxy reverso)
- Como vocês observam o sistema? (logs, métricas, alertas)

Regra: se um componente não aparece nesta lista, ele não pode aparecer no diagrama. E se aparece na lista, tem que aparecer no diagrama.

Etapa 2 — Mapear os endpoints REST

Documente os endpoints do **MVP** do projeto. Se vocês tiverem mais de 15, agrupem por recurso e detalhem pelo menos os 6 principais.

Padrões que serão cobrados:

- Substantivo no plural, sem verbo na URL → `GET /api/v1/livros` e **nunca** `GET /api/v1/getLivros`
- Versionamento no caminho (`/api/v1/...`)
- Hierarquia para sub-recursos → `GET /api/v1/usuarios/{id}/emprestimos`
- Filtros por query string → `GET /api/v1/livros?autor=asimov&page=0&size=20`
- Códigos de status coerentes (`201` para criação, `204` para exclusão, `401` vs `403`)

Modelo de tabela (veja preenchido na Seção 4.2):

Etapa 3 — Identificar o protocolo de cada ligação

Toda seta do diagrama precisa de rótulo. O formato do rótulo é:

PROTOCOLO / porta

Exemplos de rótulo válido: `HTTPS/443`, `AMQP/5672`, `TCP (PostgreSQL)/5432`, `gRPC sobre HTTP/2 - 50051`.

Tabela de referência para consulta:

Ligação típica	Protocolo	Porta padrão
Navegador / app móvel → CDN ou Load Balancer	HTTPS (HTTP/1.1 ou HTTP/2 sobre TLS 1.3)	443
Load Balancer → serviço de aplicação	HTTP/1.1	8080
Frontend → API em tempo real	WebSocket sobre TLS (WSS)	443
Serviço → serviço interno	gRPC sobre HTTP/2	50051
Aplicação → PostgreSQL	protocolo PostgreSQL sobre TCP (driver JDBC)	5432
Aplicação → MySQL	protocolo MySQL sobre TCP	3306
Aplicação → MongoDB	MongoDB Wire Protocol sobre TCP	27017
Aplicação → Redis	RESP sobre TCP	6379
Aplicação → RabbitMQ	AMQP 0-9-1	5672
Aplicação → Kafka	protocolo Kafka sobre TCP	9092
Aplicação → armazenamento de objetos (S3/MinIO)	HTTPS (API REST)	443
Aplicação → servidor de e-mail	SMTP com STARTTLS	587
Qualquer componente → resolução de nomes	DNS sobre UDP (fallback TCP)	53
Administrador → servidor	SSH	22
Prometheus → aplicação (coleta de métricas)	HTTP	8080

Atenção: na borda do sistema (o que vem da internet) o protocolo tem que ser **HTTPS**, nunca HTTP puro. Escrever `HTTP/80` numa seta que vem do navegador é erro de segurança e será descontado.

Etapa 4 — Definir as redes

Aqui vocês mostram **isolamento**. Mesmo que o projeto rode hoje só no `localhost`, o exercício é projetar como ele ficaria em um ambiente real.

Definam:

1. **A rede principal** e seu bloco CIDR (ex.: `10.0.0.0/16`)
2. **As sub-redes**, classificadas em:
 - **Pública** — recebe tráfego da internet (load balancer, gateway de saída)
 - **Privada de aplicação** — só recebe tráfego vindo da sub-rede pública
 - **Privada de dados** — só recebe tráfego vindo da sub-rede de aplicação, sem saída para a internet
3. **As regras de acesso**: quem pode entrar em cada sub-rede, vindo de onde e em qual porta

Se o projeto usa Docker Compose, vale mapear as `networks` do `docker-compose.yml` (ex.: `rede-frontend`, `rede-backend`, `rede-dados`) e indicar quais containers estão em cada uma.

Regra de ouro: banco de dados **nunca** fica em sub-rede pública. Se no diagrama de vocês o banco estiver acessível diretamente da internet.

Etapa 5 — Desenhar no draw.io

1. Acesse app.diagrams.net → **Create New Diagram** → **Blank Diagram**.
 2. Habilite os ícones de infraestrutura: **More Shapes** (canto inferior esquerdo) → marque **Networking** → **AWS 19** (ou Azure/GCP/Cisco, conforme o que o projeto usa) → **Apply**.
 3. **Desenhe as redes primeiro**. Use retângulos grandes (containers) para cada sub-rede e só depois arraste os componentes para dentro deles. Assim o componente “pertence” visualmente à rede.
 4. **Direção do fluxo**: usuário à esquerda (ou no topo), banco de dados à direita (ou embaixo). O leitor deve enxergar a requisição andando numa direção só.
 5. **Rotule todas as setas** com protocolo e porta (duplo clique sobre a linha).
 6. **Convenção de linhas** sugerida:
 - Linha contínua → chamada síncrona (requisição/resposta)
 - Linha tracejada → chamada assíncrona (mensagem em fila, evento)
 - Linha pontilhada fina → observabilidade (coleta de métricas e logs)
 7. **Legenda obrigatória**: crie uma caixa no canto explicando cores, formas e tipos de linha usados.
 8. **Camadas (opcional, mas recomendado)**: `Extras → Edit Diagram` permite editar o XML; o painel de **Layers** (`Ctrl+Shift+L`) permite separar “rede” e “aplicação” em camadas distintas.
 9. **Exportação**: `File → Export as → PDF` com a opção **Fit Page** ativada. Para inserir no documento, `Export as → PNG` com **Zoom 300%** e fundo transparente desmarcado.
 10. **Salve o .drawio**: `File → Save As → Device`.
-

Etapa 6 — Montar o documento

Monte o PDF com as seções da Seção 2. Ao escrever, faça a **verificação cruzada**:

- Todo endpoint da tabela está exposto por algum componente do diagrama?
- Toda seta do diagrama tem uma linha na tabela de protocolos?

- Todo componente do diagrama está dentro de alguma sub-rede da tabela de redes?

Se alguma resposta for “não”, o documento ainda não está pronto.

Etapa 7 — Entrega

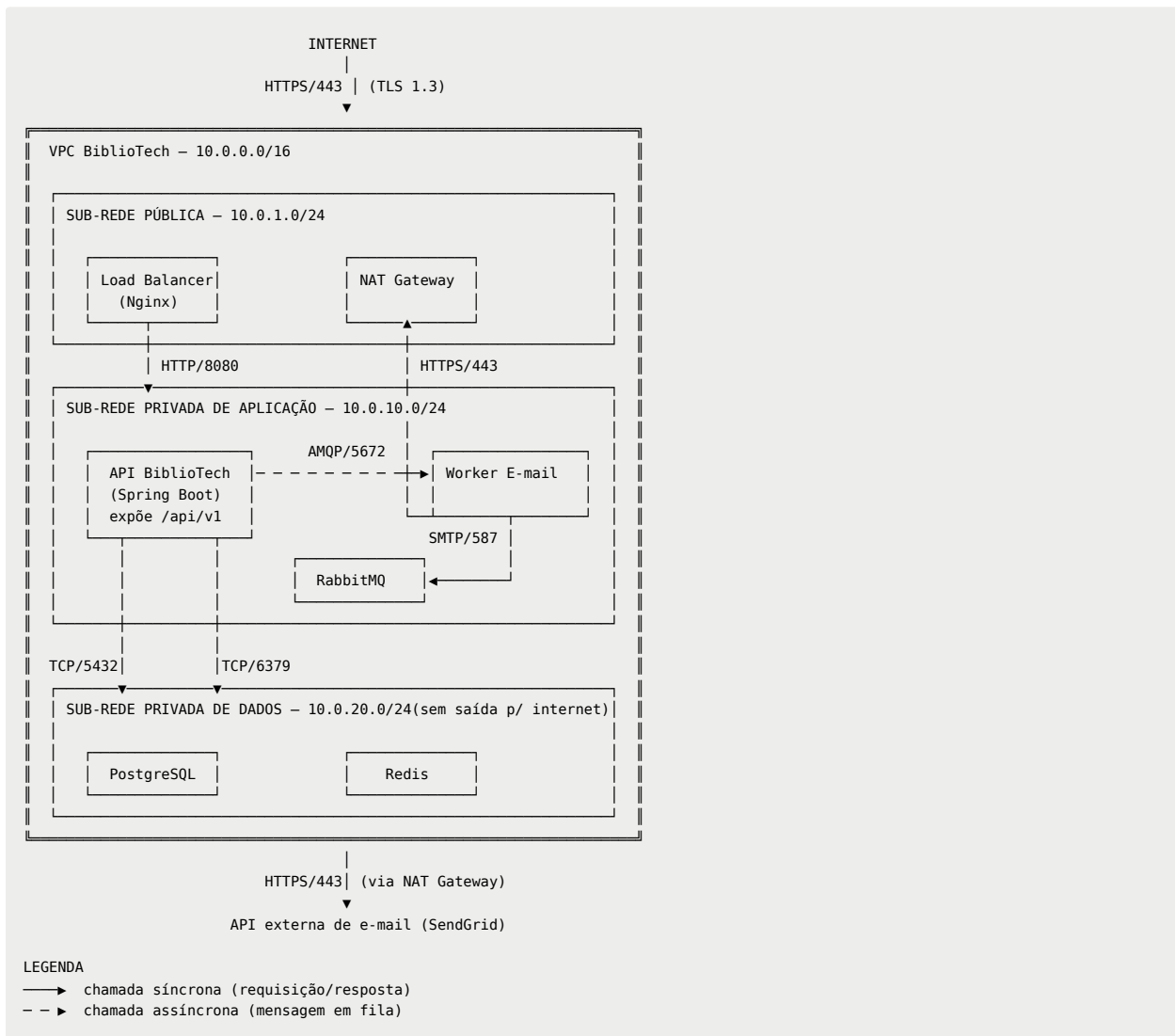
- Enviar **PDF** + **.drawio** pelo Classrom.
- Prazo: 16/10/26 - Aula subsequente a prova.
- Entregas apenas com **PDF** + **.drawio** , qualquer outra possibilidade vai ser desconsiderada e zerada a nota.

4. Exemplo completo de referência — sistema “BiblioTech”

Sistema web de empréstimo de livros de uma biblioteca universitária. Alunos consultam o acervo e reservam livros pelo navegador; bibliotecários registram empréstimos e devoluções. O sistema envia e-mail de lembrete quando a devolução se aproxima.

Este exemplo é **apenas ilustração de formato**. Não copiem os componentes — usem os do projeto de vocês.

4.1 Diagrama (representação textual do que deve ser desenhado no draw.io)



 limite de rede (VPC)
 limite de sub-rede

4.2 Tabela de endpoints REST

Método	Caminho	Descrição	Autenticação	Resposta	Status
POST	/api/v1/auth/login	Autentica e devolve token JWT	Não	{token, expiraEm}	200, 401
GET	/api/v1/livros	Lista o acervo com paginação e filtros	Não	Page<LivroResumo>	200
GET	/api/v1/livros/{id}	Detalha um livro	Não	LivroDetalhe	200, 404
POST	/api/v1/livros	Cadastra um livro no acervo	JWT (BIBLIOTECARIO)	LivroDetalhe	201, 400, 403
PUT	/api/v1/livros/{id}	Atualiza os dados de um livro	JWT (BIBLIOTECARIO)	LivroDetalhe	200, 400, 404
DELETE	/api/v1/livros/{id}	Remove um livro do acervo	JWT (BIBLIOTECARIO)	—	204, 404, 409
POST	/api/v1/emprestimos	Registra um empréstimo	JWT (BIBLIOTECARIO)	Emprestimo	201, 409
PATCH	/api/v1/emprestimos/{id}/devolucao	Registra a devolução	JWT (BIBLIOTECARIO)	Emprestimo	200, 404
GET	/api/v1/usuarios/{id}/emprestimos	Histórico de empréstimos do usuário	JWT (dono ou BIBLIOTECARIO)	List<Emprestimo>	200, 403
GET	/actuator/health	Verificação de saúde para o load balancer	Não (rede interna)	{status}	200, 503

4.3 Tabela de ligações e protocolos

#	Origem	Destino	Protocolo	Porta
1	Navegador do usuário	Load Balancer	HTTPS (HTTP/2 sobre TLS 1.3)	443
2	Load Balancer	API BiblioTech	HTTP/1.1	8080
3	Load Balancer	API BiblioTech	HTTP/1.1 (/actuator/health)	8080
4	API BiblioTech	PostgreSQL	protocolo PostgreSQL sobre TCP (JDBC)	5432
5	API BiblioTech	Redis	RESP sobre TCP	6379
6	API BiblioTech	RabbitMQ	AMQP 0-9-1	5672
7	Worker E-mail	RabbitMQ	AMQP 0-9-1	5672
8	Worker E-mail	SendGrid (externo)	HTTPS	443
9	Todos os componentes	DNS interno	DNS sobre UDP	53

4.4 Tabela de definição de redes

Rede / Sub-rede	CIDR	Tipo	Componentes	Entrada permitida	Saída permitida
VPC BiblioTech	10.0.0.0/16	—	(todas as sub-redes)	—	—

Rede / Sub-rede	CIDR	Tipo	Componentes	Entrada permitida	Saída permitida
sub-pública-a	10.0.1.0/24	Pública	Load Balancer, NAT Gateway	0.0.0.0/0 na 443	10.0.10.0/24 na 8080; internet
sub-app-a	10.0.10.0/24	Privada	API BiblioTech, Worker E-mail, RabbitMQ	10.0.1.0/24 na 8080	10.0.20.0/24 nas 5432 e 6379; internet via NAT
sub-dados-a	10.0.20.0/24	Privada isolada	PostgreSQL, Redis	10.0.10.0/24 nas 5432 e 6379	nenhuma

5. Critérios de avaliação

Critério	Pontos	O que é observado
Completude do diagrama	0,5	Todos os componentes reais do projeto aparecem; nada inventado nem faltando
Endpoints REST	0,5	Tabela completa, padrões REST respeitados, status codes coerentes
Protocolos nas ligações	1,0	Toda seta rotulada com protocolo e porta corretos; HTTPS na borda
Definição de redes	1,0	CIDR, classificação pública/privada, regras de acesso, banco isolado

6. Erros mais comuns (evitem)

1. **Seta sem rótulo.** Uma linha sem protocolo não informa nada.
2. **HTTP/80 na borda da internet.** Use HTTPS.
3. **Banco de dados em sub-rede pública.**
4. **Verbo na URL:** /api/getLivros , /api/criarUsuario , /api/deletarLivro .
5. **Imagem ilegível.** Exportem em PDF ou PNG a 300% de zoom, não deem print da tela.
6. **Trocar o diagrama de arquitetura por outro diagrama.** Não é diagrama de classes, nem DER, nem caso de uso.
7. **Ícone bonito sem legenda.** Se ninguém entende o símbolo, ele não comunica.
8. **Componentes que existem no diagrama mas não no projeto** ("Kafka" porque parece profissional, sem estar no código).
9. **Esquecer o que é assíncrono.** Fila e evento não são requisição/resposta; use linha tracejada.
10. **Entregar só o PDF, sem o .drawio .**